Project - Submission Sheet

Software Testing - Input Space Partitioning

Names: [Arpita]
Group: [4]

Software Testing - Summer 2024 - June 15, 2025 - Sibylle Schupp / Daniel Rashedi

Task 2 - Input Space Partitioning

a) Explain why the selected methods are suitable for the input domain model approach (1-2 sentences each)

Answer:

For input space partitioning *split_version* function is a suitable one. This functions takes a string input like '1.2.3' and gives output as a tuple of integers (1,2,3).The reasons for selecting this function for ISP is given below:

- It takes structured input.
- It's behavior depends on the format and content of the input.
- Input characteristics are available and also creating equivalent class is possible.
- the 2 main properties like completeness and disjoint can also be preserved.

b) Identify the input domain of your methods (2-3 sentences each)

Answer:

Input domain can be considered as following:

- Valid semantic versions.
- Partial versions like '1.2'
- Partialy invalid version like '1.2.a'
- Totally invalid version like 'abc'
- Invalid version using special characters like'1-2-3'

c) Similar to Homework 02, gather the results of the following sub-tasks:

1) Identify reasonable characteristics of the possible input data
2) Derive sets of equivalence classes from these characteristics (Note: At least one of your partitions must consist of a number of blocks ≥ 3. Also make sure that you have enough characteristics and corresponding blocks to derive 12 distinct tests from them later on.)

Answer:

Characteristic	**Block 1**	**Block 2**	**Block 3**	**Block 4**
q1 = Number of components	<3	3	>3	null
q2 = Component parseability	All valid integers	Some valid integers	None valid	Empty string
q3 = Separator used	Dot (.)	Other (-, /)	None	Mixed
q4 = Output tuple length	Empty	<3	3	>3
q5 = Input type	String	Integers	tuple	list

d) Can you identify any non-valid combinations of blocks from your characteristics? (1-2 sentences each)

Answer:

There are several non-valid combination of blocks exists. They are given below:

- . **Block 4 from q1 + Block 4 from q4:**The input has no components, so the output tuple cannot contain more than 3 elenents.
- . **Block 3 from q1 + Block 4 from q2:**An empty string cannot contain any components, so it cannot simultaneously belong to the block 3 of q1.
- . **Block 1 from q3 + Block 2 from q5:**A non-string input such as an integer cannot contain any separator like a dot or dash. Since separators only apply to strings, this combination is invalid.
- . **Block 1 from q2 + Block 4 from q5:**A list input is not parsed as a version string. So, it's invalid to classify a list under "All valid integers."
- . **Block 4 from q1 + Block 2 from q2:**If the input has no components (null), then it cannot contain some valid integers. These blocks contradict each other and represent an invalid combination.

e) Derive (at least) one representative value for each equivalence class. Which strategy did you use for the value selection?

Answer:

Block	**Representative Value**	**Strategy Used**
<3	`"1.2"`	Typical 2-part input
3	`"1.2.3"`	Nominal valid input
>3	`"1.2.3.4"`	Slightly above valid range
null	`""`	Boundary/empty case

Table 1: Representative values and strategies for `q1 = Number of Components`

Block	**Representative Value**	**Strategy Used**
All valid integers	`"4.5.6"`	Typical, clean data
Some valid integers	`"1.b.3"`	Mixed-valid format
None valid	`"x.y.z"`	Invalid format
Input is empty	`""`	Boundary case

Table 2: Representative values and strategies for `q2 = Component Parseability`

Block	**Representative Value**	**Strategy Used**
Dot (.)	`"2.3.4"`	Standard case
Other	`"2-3-4"`	Structural variation
None	`"123"`	No separator
Mixed	`"1.2-3"`	Edge case (confusing input)

Table 3: Representative values and strategies for `q3 = Separator Used`

Block	Representative Value	Strategy Used
Empty	`"a.b.c"`	No part is parsable
<3	`"1.2.a"`	Stops after two parts
3	`"1.2.3"`	Valid full parse
>3	N/A	Not applicable for current function logic

Table 4: Representative values and strategies for `q4 = Output Tuple Length`

Block	Representative Value	Strategy Used
String	`"1.2.3"`	Normal expected input
Integer	`123`	Type mismatch test
Tuple	`(1,2,3)`	Type mismatch test
List	`["1", "2", "3"]`	Type mismatch test

Table 5: Representative values and strategies for `q5 = Input Type`

f) Combine the selected values from the partitions of each input argument to complete test vectors using `hypothesis`. Document only the excerpt of your code responsible for generation of value combinations.

Answer:

[Your Answer]

```python
from hypothesis import given, strategies as st

component_variants = st.sampled_from(["1.2", "1.2.3", "1.2.3.4", ""])
parseability_variants = st.sampled_from(["4.5.6", "1.b.3", "x.y.z", ""])
separator_variants = st.sampled_from(["2.3.4", "2-3-4", "123", "1.2-3"])
tuple_length_variants = st.sampled_from(["a.b.c", "1.2.a", "1.2.3"])
input_type_variants = st.sampled_from(["1.2.3", 123, (1, 2, 3), ["1", "2", "3"]])

test_vectors = st.tuples(
    component_variants,
    parseability_variants,
    separator_variants,
    tuple_length_variants,
    input_type_variants
)
@given(test_vectors)
def test_show_tuples(values):
    print(values)
```

Test Vector	q1	q2	q3	q4	q5
1	1.2	4.5.6	1.2-3	1.2.3	`['1', '2', '3']`
2	1.2.3	1.b.3	2.3.4	1.2.3	123
3	*empty*	1.b.3	2.3.4	1.2.3	1.2.3
4	1.2	*empty*	2.3.4	1.2.3	`['1', '2', '3']`
5	1.2.3	x.y.z	2.3.4	1.2.a	`(1, 2, 3)`
6	1.2.3.4	x.y.z	2.3.4	1.2.a	1.2.3

g) Create Python tests for your method, which trigger test runs with the selected test vectors / blocks, and assert each individual output. For the submission, merge all these tests into one `*.py` file.

Answer:

```python
            import pytest
from typing import Tuple

# Function to test
def split_version(version: str) -> Tuple[int, ...]:
    parts = []
    for part in version.split('.')[:3]:
        try:
            parts.append(int(part))
        except ValueError:
            break
    return tuple(parts)

# 6 test cases derived from earlier test vectors
test_cases = [
    ("1.2.3", (1, 2, 3)),
    ("4.5.6", (4, 5, 6)),
    ("1.b.3", (1,)),
    ("1.2-3", (1,)),   # "2-3" is invalid int
    ("1.2.a", (1, 2)),
    ("123", (123,)),
]

@pytest.mark.parametrize("version_str, expected", test_cases,
    ids=[f"{v[0]} -> {v[1]}" for v in test_cases] )
def test_split_version(version_str, expected):
    result = split_version(version_str)
    assert result == expected, f"For input '{version_str}', expected {expected}, got {result}"
```

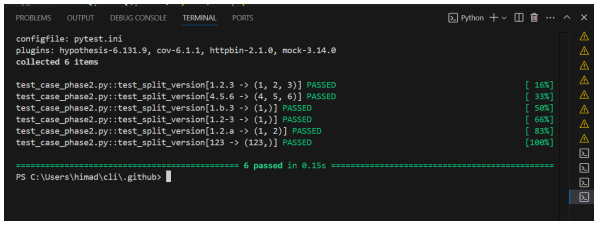

Figure 1: Test case result generated using test vectors